CO3133 · DEEP LEARNING AND ITS APPLICATIONS

Comparing Five Architectures on Fashion-MNIST

Linear · MLP · CNN · LSTM · Transformer

Assignment 1 · Team G-09 · Class CC01 · HCMUT

How do different architectures, and the way an image is fed to them, affect Fashion-MNIST classification?

Nguyen Vu Long (2352696) · Le Phuoc Vu (2353341) · Nguyen Hoang Quoc (2353027)
Lecturer: Le Thanh Sach



Problem and objectives

How do architecture and input layout affect classification?

- Task: supervised, single-label, 10-class classification of a 28×28 grayscale image (Eq. 1: logits $z = f_\theta(T(X))$, prediction = argmax)
- Motivation: sorting product images in an online catalogue; reliability depends on the data, the split, the image representation and the cost, not only on the model
- Goal: an end-to-end PyTorch pipeline that compares the five mandatory models (Linear, MLP, CNN, LSTM, Transformer) on identical splits
- Measures: accuracy, macro-F1, training and inference time, parameters, learning curves and errors; accuracy alone is not enough

01

Performance: test accuracy and macro-F1 of each model

02

Efficiency: parameters, training time and forward time

03

Learning behaviour: loss curves and checkpoint choice

04

Errors: per-class F1, confusion patterns, shared errors

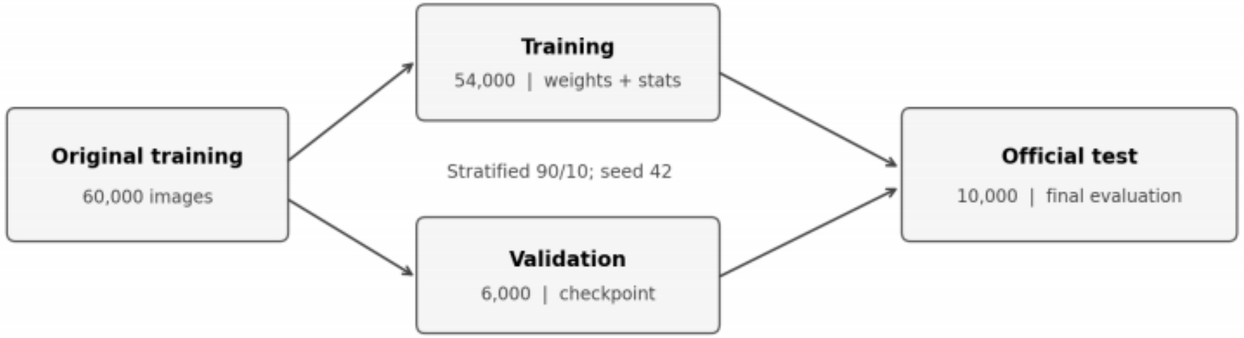
Pipeline (Eq. 1): image X (28×28) → preprocessing T(X) → classifier fθ → 10 logits → argmax = predicted class

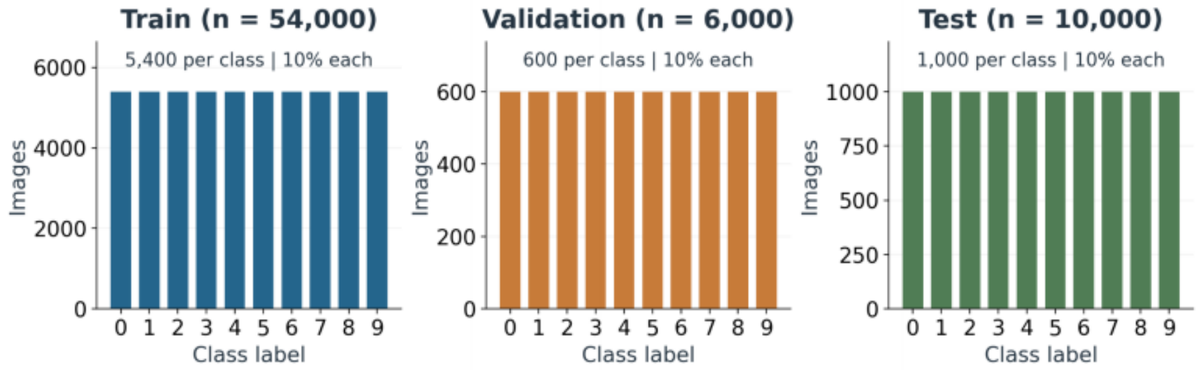
Dataset and data split

70,000

official images: 60,000 training + 10,000 test · 10 classes

- Fashion-MNIST: 28×28 unsigned 8-bit pixels (0–255), one integer label per image, 10 clothing classes
- Stratified 90/10 split of the training set, seed 42: 54,000 train and 6,000 validation images; the 10,000 test images stay separate
- Roles: train fits weights and normalization, validation selects the checkpoint, test is used only for the final evaluation
- Leakage check: a SHA-256 audit finds 0 duplicates within or across splits (near-duplicates cannot be detected)
- Caveat: low-resolution grayscale images do not fully represent real product photos

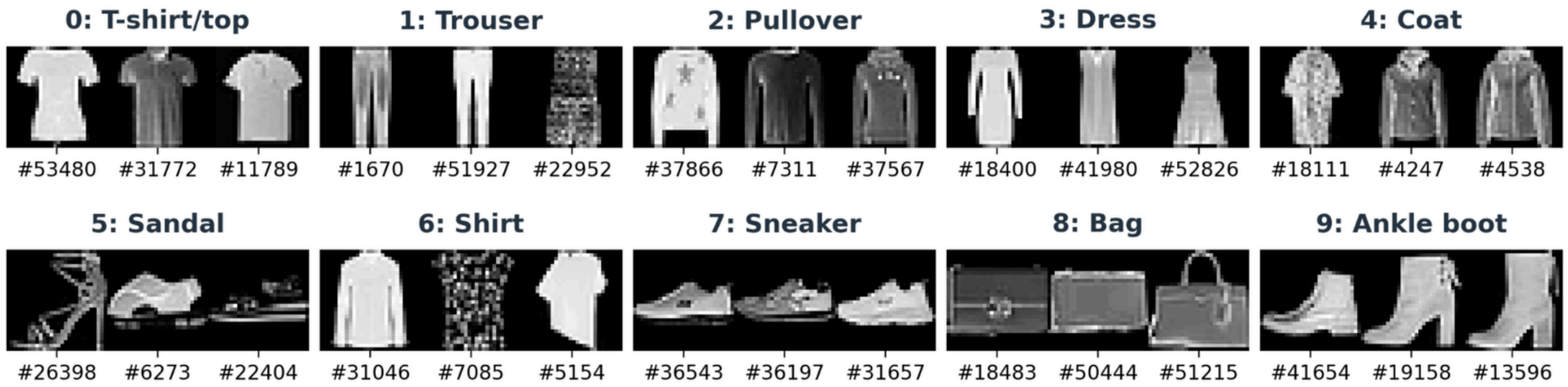




Exploratory data analysis

What the EDA shows

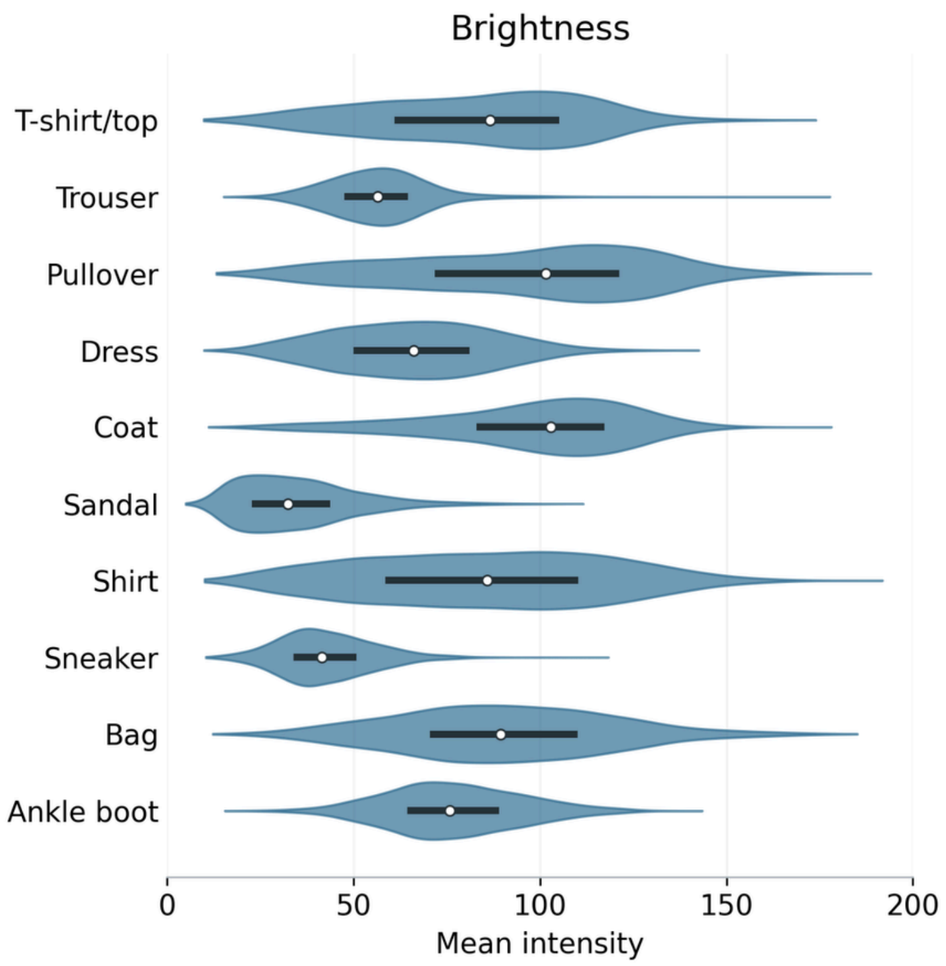
- Perfectly balanced: 5,400 / 600 / 1,000 images per class in train / validation / test, so no rebalancing is needed
- 50.21% of training pixels are exactly 0; mean intensity 72.98, standard deviation 90.06; no image is blank
- Brightness and pixel coverage overlap heavily across classes (Coat brightest at 98.15, Sandal darkest at 34.87), so they cannot separate classes alone
- Coat and Shirt class-average images have cosine similarity 0.9873: upper-body garments look alike

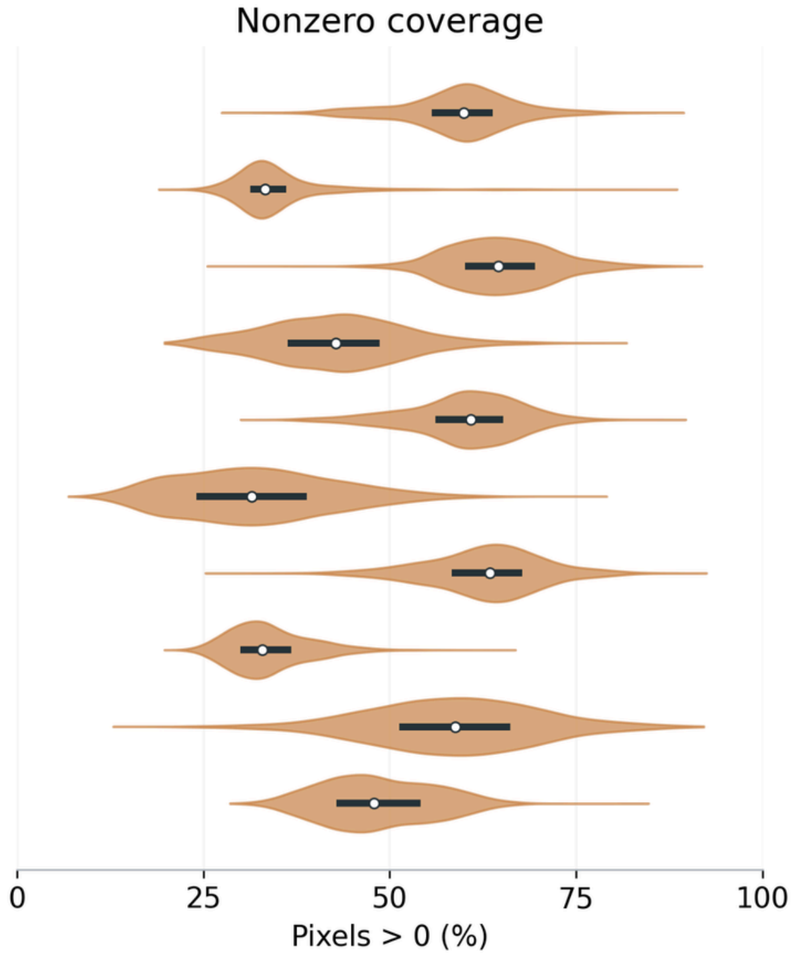


Exploratory data analysis

What the EDA shows

- Perfectly balanced: 5,400 / 600 / 1,000 images per class in train / validation / test, so no rebalancing is needed
- 50.21% of training pixels are exactly 0; mean intensity 72.98, standard deviation 90.06; no image is blank
- Brightness and pixel coverage overlap heavily across classes (Coat brightest at 98.15, Sandal darkest at 34.87), so they cannot separate classes alone
- Coat and Shirt class-average images have cosine similarity 0.9873: upper-body garments look alike

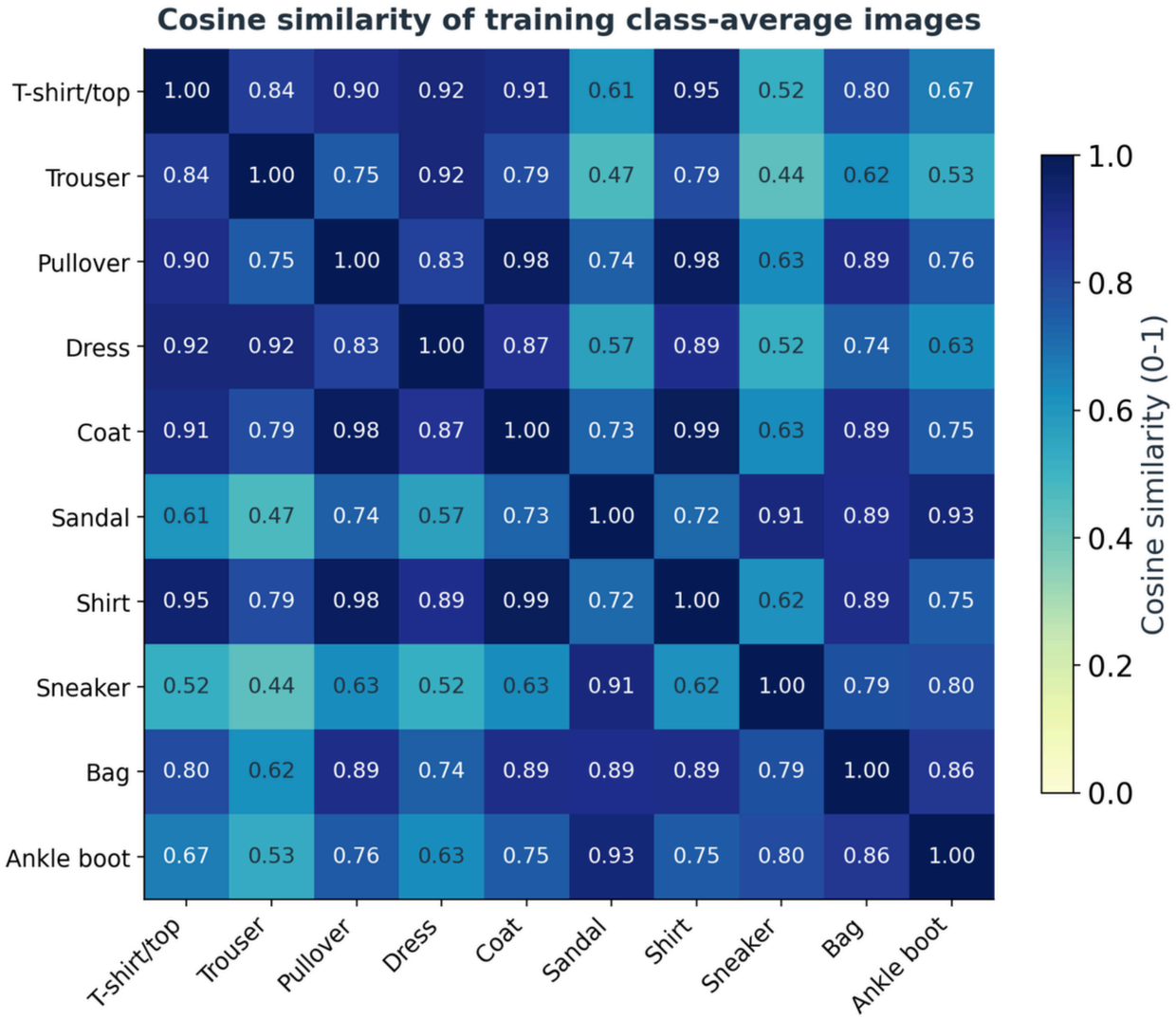




Exploratory data analysis

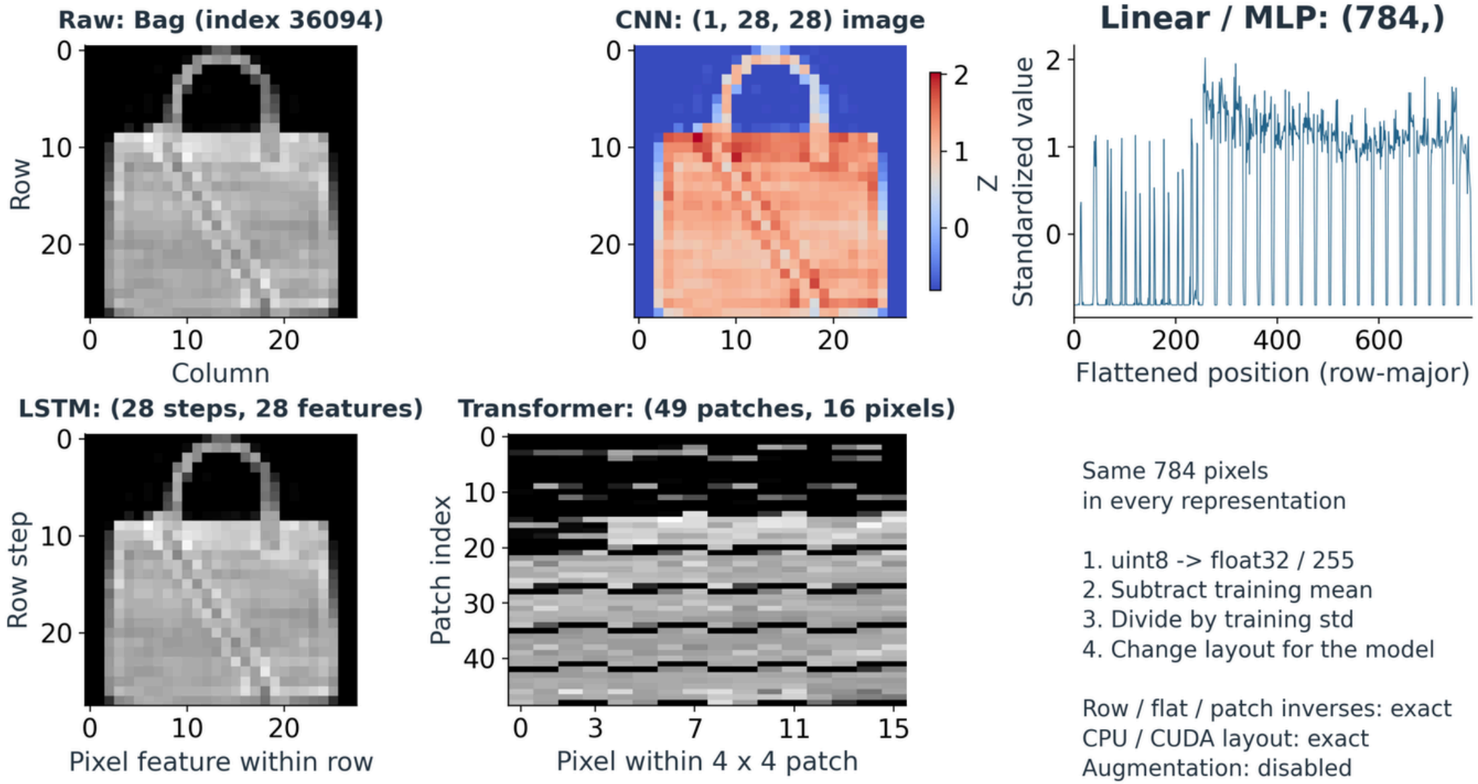
What the EDA shows

- Perfectly balanced: 5,400 / 600 / 1,000 images per class in train / validation / test, so no rebalancing is needed
- 50.21% of training pixels are exactly 0; mean intensity 72.98, standard deviation 90.06; no image is blank
- Brightness and pixel coverage overlap heavily across classes (Coat brightest at 98.15, Sandal darkest at 34.87), so they cannot separate classes alone
- Coat and Shirt class-average images have cosine similarity 0.9873: upper-body garments look alike



Preprocessing and input representations

- Pixels divided by 255, then standardized with the training-set mean 0.286209 and std 0.353160; validation and test reuse the same constants, so nothing leaks
- No resizing, augmentation or sample removal: every model sees the same 784 values
- One image, four layouts: flat vector (Linear, MLP), 1×28×28 grid (CNN), 28 rows of 28 features (LSTM), 49 patches of 16 pixels (Transformer)
- Verified: mean ≈ 0 and std ≈ 1 after standardization (deviation below 10^{-6}); flatten, row and patch conversions lose no pixel; CPU and CUDA agree



Same 784 pixels
in every representation

1. uint8 -> float32 / 255
2. Subtract training mean
3. Divide by training std
4. Change layout for the model

Row / flat / patch inverses: exact
CPU / CUDA layout: exact
Augmentation: disabled

The five models

LINEAR 7,850 parameters	MLP 235,146 parameters	CNN 421,642 parameters	LSTM 82,186 parameters	TRANSFORMER 71,946 parameters
Input: flat vector of 784 values. A single linear map from 784 values to 10 logits. Neighbouring pixels get no special treatment.	Input: flat vector of 784 values. Hidden layers of 256 and 128 units with ReLU and dropout 0.2, then 10 outputs. Also blind to pixel neighbourhoods.	Input: 1×28×28 grid. Two 3×3 convolution blocks (32 then 64 feature maps) with ReLU and 2×2 max-pooling, then dense 128 and 10 outputs. The dense layer holds most of the parameters.	Input: 28 rows read as a sequence. Each row is one time step with 28 features; 128 hidden units; the last hidden state feeds the classifier.	Input: 49 patch tokens. Each token is projected to 64 dimensions and given a learned position; 2 encoder blocks with 4 attention heads; mean over tokens, then classifier.

fewer assumptions **stronger structure**

Why compare them: each model assumes a different structure in the image (its inductive bias), so the comparison shows which assumption pays off. The cards run from fewer assumptions (flat vector) to stronger structure (local filters, row order, patch attention).

Linear: one weighted score per class

The simplest baseline connects all 784 pixels directly to ten output scores.

Flat pixels

784 values

→

Class weights

10 sets of 784

→

Weighted sum

$z = Wx + b$

→

Prediction

10 logits, argmax

How one score is calculated

For each class, multiply every pixel by its learned weight, add the products, then add a bias.

W has shape 10 × 784. The bias has 10 values: 7,840 + 10 = 7,850 parameters.

What this baseline can learn

Each class learns a position-specific pixel pattern. There is no hidden layer or nonlinear feature extractor.

All models train on raw logits with cross-entropy. Softmax is unnecessary before the loss or argmax.

7,850 parameters	Accuracy 84.26%	Macro-F1 0.8417	Selected epoch 9

Shapes describe one image. Scores and epochs: original PDF / comparison.json, seed 42.

MLP: nonlinear combinations of pixels

Two hidden layers combine the flat input into increasingly abstract features.



Inside each hidden layer

A dense layer mixes all its inputs. ReLU keeps positive activations and sets negative ones to zero.

Dropout 0.2 follows each ReLU during training. Validation and testing disable dropout.

What changes from Linear

Nonlinear hidden layers can combine pixel evidence in ways a single linear map cannot.

The MLP keeps every pixel, but uses no shared local filters. Its 4.24-point accuracy gain also comes with greater capacity.

235,146 parameters	Accuracy 88.50%	Macro-F1 0.8846	Selected epoch 8

Shapes describe one image. Scores and epochs: original PDF / comparison.json, seed 42.

CNN: shared filters learn local patterns

Convolution keeps the image grid, while pooling reduces its spatial size.

Image grid	→	Conv + pool	→	Conv + pool	→	Flatten + dense	→	Class scores
1 × 28 × 28		32 × 14 × 14		64 × 7 × 7		3,136 to 128		10 logits

Inside the two convolution blocks

Each block uses a 3 × 3 convolution, padding 1, ReLU, then 2 × 2 max pooling.

Convolution preserves spatial size. Pooling halves it: 28 to 14 to 7. Shared filters reuse a detector at different locations.

The classifier holds most weights

Flatten 64 × 7 × 7 features, then dense 128, ReLU, dropout 0.2, and dense 10.

The first dense layer has 401,536 parameters, about 95% of the total. The best score here does not isolate a pure convolution effect.

421,642 parameters	Accuracy 91.94%	Macro-F1 0.9189	Selected epoch 10

Shapes describe one image. Scores and epochs: original PDF / comparison.json, seed 42.

LSTM: a memory updated one row at a time

The 28 image rows form a spatial sequence, read from top to bottom.

Row sequence
28 steps × 28 pixels

→

LSTM layer
128 hidden units

→

Final hidden state
128 values, h at row 28

→

Readout
Dropout 0.2 + dense 10

How the memory changes

At each row, the forget gate controls old memory, the input gate controls new content, and the output gate exposes the hidden state.

Hidden and cell states start at zero for each image. The same LSTM weights process every row.

Exactly which state is classified

A single forward layer reads all 28 rows. Only its final hidden state enters the classifier.

The cell state carries memory internally. There is no averaging of row outputs. The 88.48% score is only two test images behind MLP.

82,186 parameters	Accuracy 88.48%	Macro-F1 0.8835	Selected epoch 9

Shapes describe one image. Scores and epochs: original PDF / comparison.json, seed 42.

Transformer: patches exchange information

Self-attention connects all 49 patches, using learned positions to retain layout.

Patch sequence
49 patches × 16 pixels

→

Embed + position
49 tokens × 64

→

2 encoder blocks
4 heads per block

→

Normalize + mean
49 tokens become 64

→

Readout
Dropout + dense 10

Inside each encoder block

Each attention head compares queries and keys, then mixes values from all patches. Each head uses 16 features.

Pre-norm residual sublayers wrap attention and a 64-to-128-to-64 feedforward network with GELU. Dropout is 0.2.

From an image to one prediction

Split the image into a 7 × 7 grid of 4 × 4 patches in raster order. Project 16 pixels to 64 features, add positions, then apply dropout.

After the blocks, layer-normalize and average all tokens. No CLS token or causal mask. Training starts from scratch.

71,946 parameters	Accuracy 85.55%	Macro-F1 0.8524	Selected epoch 10

Shapes describe one image. Scores and epochs: original PDF / comparison.json, seed 42.

Experimental setup and fairness

SHARED TRAINING PROTOCOL

- Adam, learning rate 0.001, batch size 128, 10 epochs
- Cross-entropy loss on logits, seed 42
- No scheduler, weight decay or gradient clipping
- Same stratified 54,000 / 6,000 / 10,000 split and the same normalization constants for every model

CHECKPOINT AND TEST

- Checkpoint = the epoch with the lowest validation loss
- The test set is evaluated once, for that fixed checkpoint
- Reported: accuracy, macro-F1, test loss, number of errors and per-class results

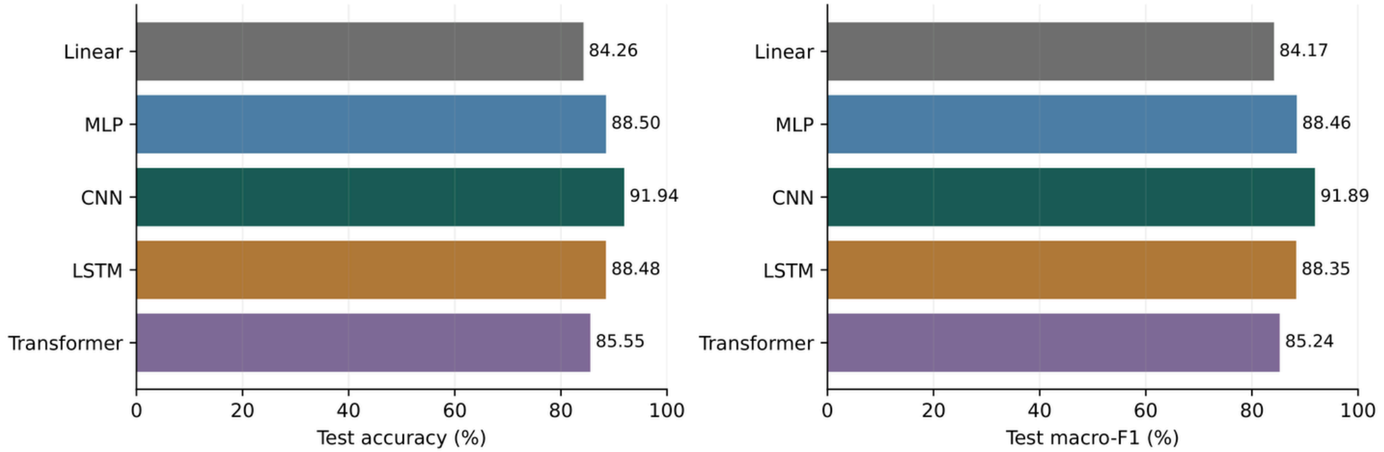
ENVIRONMENT AND TIMING

- NVIDIA GTX 1660 SUPER (6 GiB), Windows 11, Python 3.12.10, PyTorch 2.9.1+cu128
- Training time = the 10-epoch loop
- Forward time = mean of 100 batch-128 calls after 10 warm-ups, data loading excluded

Protocol in one line: train, validate every epoch, keep the lowest validation loss, test once.

Main results

- The CNN leads on every headline metric: 91.94% accuracy, macro-F1 0.9189, test loss 0.2366, 806 errors
- Against the MLP it gains 3.44 points and cuts errors from 1,150 to 806 (29.9% fewer); against Linear it gains 7.68 points
- Nonlinearity matters: the MLP beats Linear by 4.24 points (88.50% vs 84.26%) on the same flat input
- MLP and LSTM are tied (88.50% vs 88.48%, two images apart, one seed each), so there is no stable ranking; the Transformer reaches 85.55%

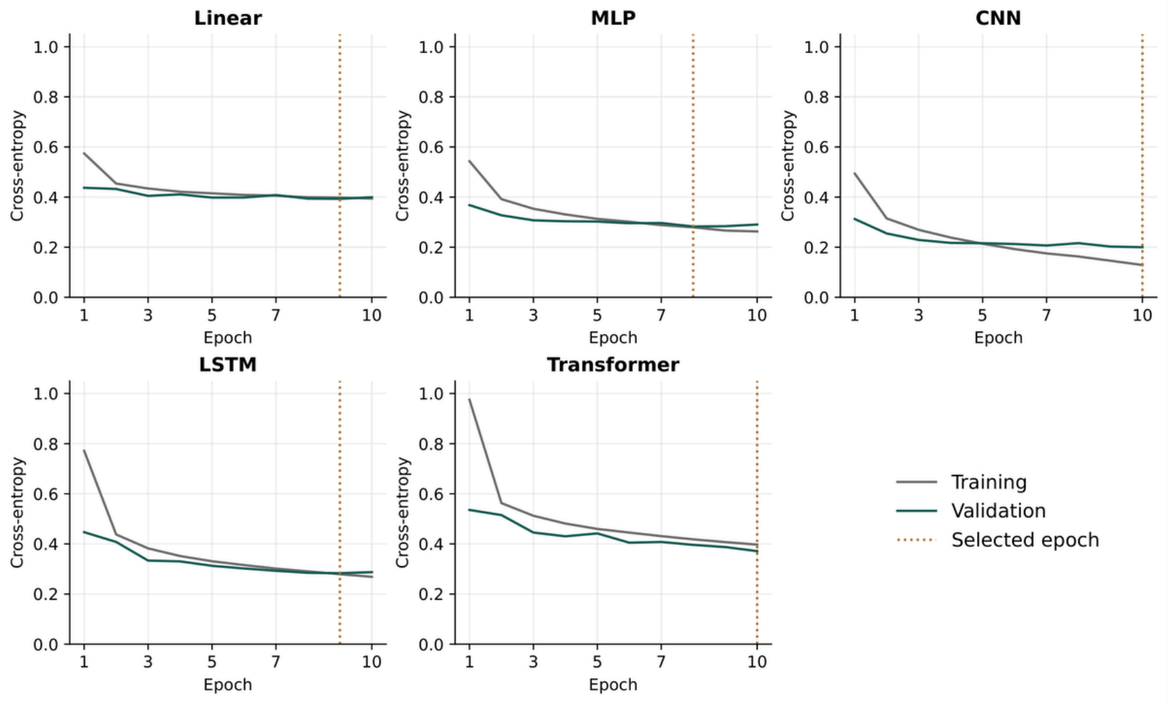


Test accuracy by model
Linear 84.26% · MLP 88.50% · CNN 91.94% · LSTM 88.48% · Transformer 85.55%
Say macro-F1 as well: models are not ranked by accuracy alone.

Model	Epoch	Val. acc.	Test acc.	Test F1	Test loss	Errors
Linear	9	86.48	84.26	0.8417	0.4454	1,574
MLP	8	89.87	88.50	0.8846	0.3251	1,150
CNN	10	93.28	91.94	0.9189	0.2366	806
LSTM	9	89.80	88.48	0.8835	0.3254	1,152
Transformer	10	86.43	85.55	0.8524	0.4085	1,445

Learning behaviour and computational cost

- Selected checkpoints: epoch 10 for CNN and Transformer, 9 for Linear and LSTM, 8 for MLP; the MLP's validation loss rises from 0.2825 (epoch 8) to 0.2906 (epoch 10)
- CNN: training accuracy 95.19% vs validation 93.28%; the Transformer's validation loss is still falling at epoch 10
- Forward time per batch of 128 on CUDA: Linear 0.060 ms, MLP 0.326, CNN 1.074, LSTM 1.189, Transformer 2.061; training takes 90.85 s (Linear) to 139.45 s (Transformer)
- Parameter count is not speed: LSTM and Transformer have fewer parameters than the MLP but slower forward passes
- Times are per batch of 128 from single runs, not single-image latency

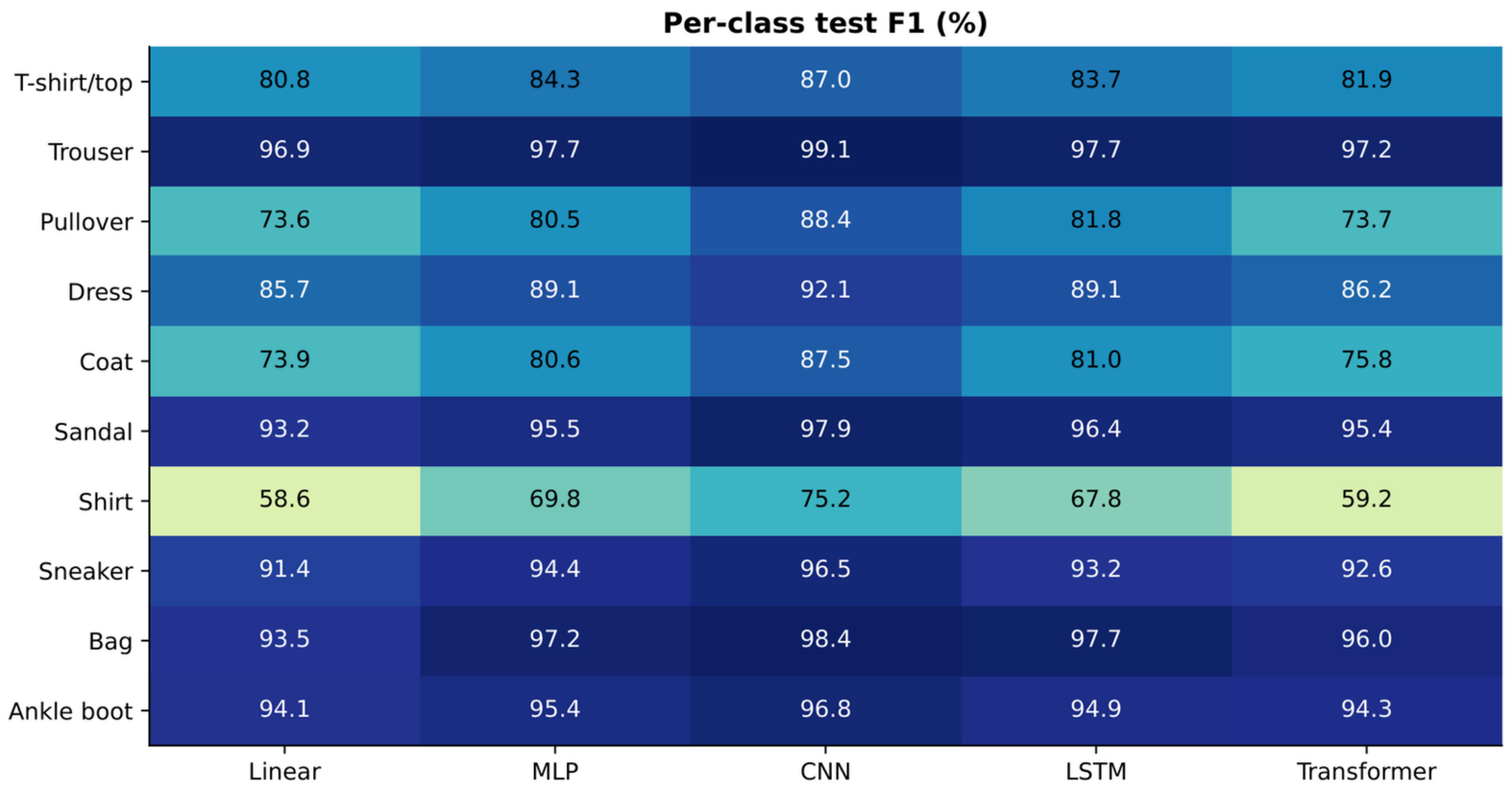


Model	Val. loss, epoch 1	Val. loss, epoch 10	Train acc. (%)	Val. acc. (%)
Linear	0.4373	0.3993	86.17	86.15
MLP	0.3680	0.2906	90.19	89.65
CNN	0.3124	0.2001	95.19	93.28
LSTM	0.4469	0.2874	90.15	89.40
Transformer	0.5355	0.3713	85.47	86.43

Model	Parameters	Train (s)	Train device	Forward (ms)	Test pass (s)
Linear	7,850	90.85	CUDA	0.060	1.110
MLP	235,146	96.69	CUDA	0.326	1.410
CNN	421,642	105.03	CUDA	1.074	1.382
LSTM	82,186	122.12	CUDA	1.189	1.237
Transformer	71,946	139.45	CUDA	2.061	1.595

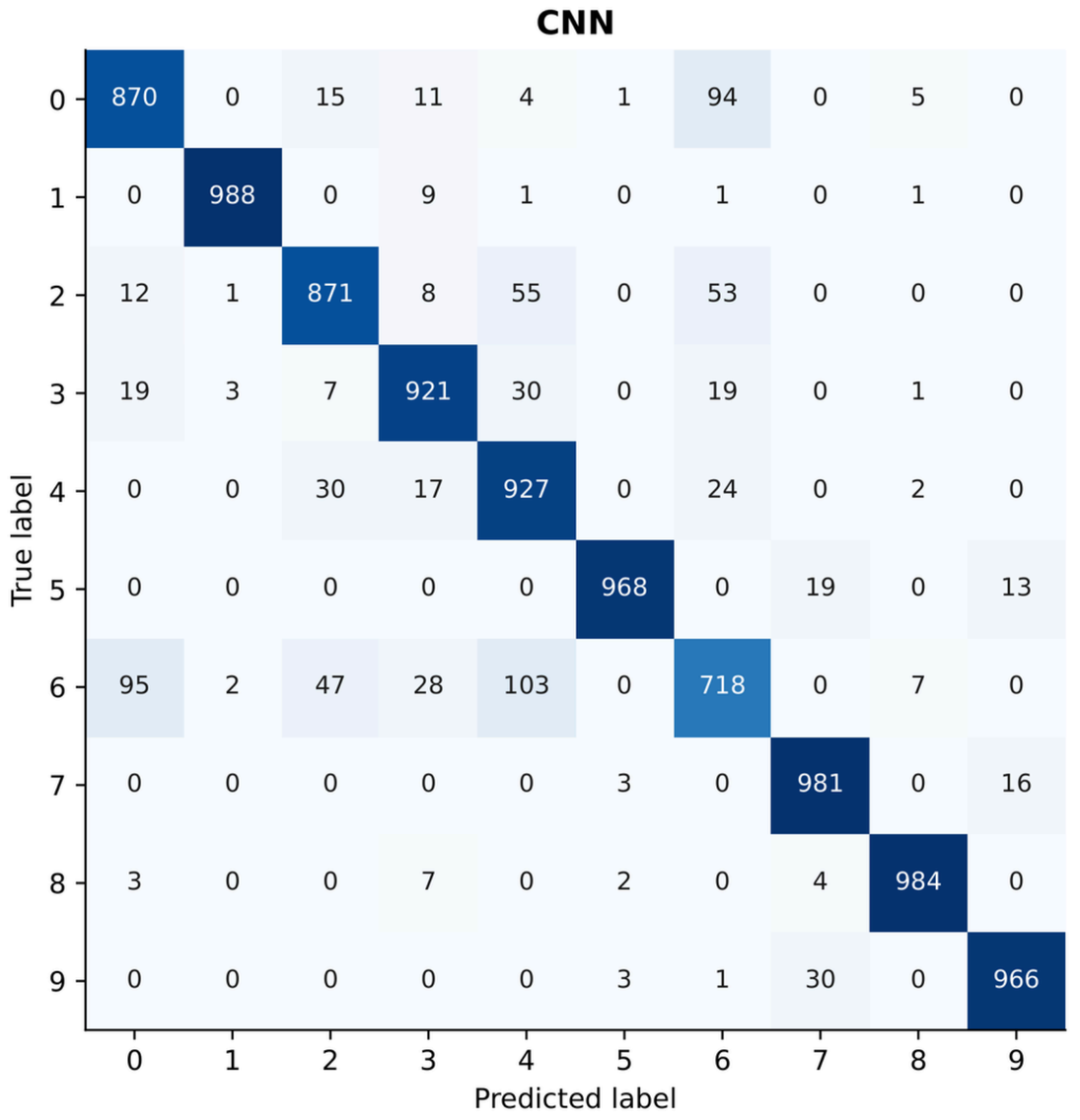
Error analysis

- Shirt is the hardest class for every model: F1 58.59 (Linear), 69.76 (MLP), 75.18 (CNN), 67.84 (LSTM), 59.20 (Transformer); for the CNN, Trouser 99.10, Bag 98.40 and Sandal 97.93 are easiest
- 532 of the CNN's 806 errors (66.0%) are among T-shirt/top, Pullover, Coat and Shirt; Shirt alone has 282 errors
- Asymmetric confusion: 103 Shirt images become Coat, but only 24 Coat images become Shirt; Coat recall is 92.7% and precision 82.77%
- Models err differently: the CNN fixes 566 MLP errors while the MLP fixes 222 CNN errors; 389 images are wrong for all five models



Error analysis

- Shirt is the hardest class for every model: F1 58.59 (Linear), 69.76 (MLP), 75.18 (CNN), 67.84 (LSTM), 59.20 (Transformer); for the CNN, Trouser 99.10, Bag 98.40 and Sandal 97.93 are easiest
- 532 of the CNN's 806 errors (66.0%) are among T-shirt/top, Pullover, Coat and Shirt; Shirt alone has 282 errors
- Asymmetric confusion: 103 Shirt images become Coat, but only 24 Coat images become Shirt; Coat recall is 92.7% and precision 82.77%
- Models err differently: the CNN fixes 566 MLP errors while the MLP fixes 222 CNN errors; 389 images are wrong for all five models



Limitations and next steps

LIMITATIONS

- One training seed per model: the 0.02-point MLP–LSTM gap cannot be called a real difference
- Capacity is not controlled (CNN 421,642 parameters vs Linear 7,850) and no ablation separates convolution from size
- Ten-epoch budget and no hyperparameter search: the Transformer was still improving, so its score is not a verdict on attention
- Timings come from separate sessions; Fashion-MNIST has only simple grayscale images

NEXT STEPS

- Several seeds, reporting the mean and its variation
- Longer training schedules chosen on the validation set
- Ablations: CNN width, LSTM readout, patch size
- Then augmentation and calibration

WHAT WE CAN CLAIM

The CNN leads in this setup (91.94%, macro-F1 0.9189)
Shirt is the hardest class for all five models

WHAT WE CANNOT CLAIM YET

That convolution itself causes the gain
That MLP and LSTM really differ, or that attention is worse than convolution

Conclusion and discussion

01 In our configurations the CNN is best (91.94%, macro-F1 0.9189, 806 errors vs 1,150 for the MLP), but no ablation shows that convolution is the cause
MLP and LSTM tie at about 88.5%, the MLP with a shorter forward time (0.326 vs 1.189 ms); the Transformer reaches 85.55% under a short budget; Shirt and other upper-body garments stay hardest

Q1 The CNN wins but also has the most parameters, and the study has no ablation. Which controlled experiment (for example matched parameter counts, or removing pooling) would show whether the gain comes from local spatial structure or from model size?

Q2 For an online catalogue, would you deploy the CNN (91.94%, 1.074 ms per 128-image batch) or the MLP (88.50%, 0.326 ms), and which measure would decide it: accuracy, macro-F1 or Shirt recall (CNN: 71.80%)?

Questions?